

The SaResQ Perception Model

Architecture, training methodology, and deployment validation

DOCUMENT	DATE	MODEL	COMMIT	TARGET
Technical report v1.1	11 September 2026	v8n_p3_thermalmix2_640	4fe9173	Raspberry Pi 4, 1 GB

This report documents the perception stack that SaResQ flies: what it is, why this architecture was chosen over the alternative that was built and measured alongside it, how it was trained and on what, and what validation was performed before deployment. Every number here was measured on this hardware; none are quoted from literature. Where a result is weak or unproven, it is labelled as such.

Executive summary

SaResQ detects human beings from a small drone using a **thermal-first cascade**. A cheap statistical test on a 32×24 thermal frame nominates a handful of candidate regions; a convolutional detector then examines only those regions; a scene classifier supplies disaster context; and a calibrated log-odds ledger fuses the evidence into a single probability per candidate. The architectural bet is that **the expensive model should never see the whole frame** — which is what allows a 3.2 MB network to run usefully on a 1 GB single-board computer.

COMPONENT	ARCHITECTURE	SIZE	MEASURED RESULT
Person detector	YOLOv8n, P3–P5 head	3.23 MB	Person AP₅₀ 0.850 (HIT-UAV) / 0.544 (RGBT-thermal)
Scene hazard classifier	MobileNetV2, transfer	2.74 MB	95.2% accuracy, 5 classes
Evidence fusion	Logistic regression, 18 features	1.3 KB	ROC AUC 0.670 (weak)

The three findings that matter most

- P2 lost to P3.** The stride-4 "small object" head — the intuitive choice for detecting tiny people — was built, trained identically, and *lost every object-size bin at twice the compute*. Evidence beat intuition.
- Full INT8 quantisation was destroying the model silently.** The file that would have flown scored **31.9% of the reference model's accuracy**. It loaded, ran fast, and returned plausible boxes. Switching to w8a32 recovered it to **99.2%** at the same file size.
- One dataset's visible imagery was unusable** — RGBTDronePerson was captured at night. Three training runs failed against it before the data, rather than the training, was examined.

Reading this report · the terms used

This section defines every piece of jargon that follows. Nothing later in the document assumes you have read it elsewhere.

Measuring a detector

TERM	WHAT IT ACTUALLY MEANS
IoU Intersection over Union	How much a predicted box overlaps the true box. Divide the shared area by the combined area. 1.0 is a perfect match, 0 is no overlap. It is the yardstick every other metric is built on.
AP₅₀ Average Precision at IoU 0.5	The standard detection score. A prediction counts as correct if it overlaps the true box by at least 50%. AP then summarises the precision/recall curve across all confidence thresholds into one number between 0 and 1. "0.85" means the detector finds and correctly boxes about 85% of the people , allowing for the trade between missing some and raising false alarms.
AP₅₀₋₉₅	The strict version: the same score averaged over ten overlap requirements from 50% up to 95%. It punishes loose boxes. A detector with high AP ₅₀ but low AP ₅₀₋₉₅ <i>finds</i> objects well but <i>outlines</i> them imprecisely — which is exactly this model's profile, and an acceptable one here.
mAP mean Average Precision	AP averaged across all classes. Misleading here, because one junk class (<code>OtherVehicle</code>) sits near 0.05 and drags the mean down. This report quotes Person AP₅₀, never mAP.
Precision / Recall	<i>Precision</i> : of everything flagged, how much was real. <i>Recall</i> : of everything real, how much was flagged. Search-and-rescue weights recall far above precision — a false alarm costs a few seconds of an operator's attention; a miss can cost a life.
ROC AUC	For a yes/no scorer: the probability it ranks a true positive above a random negative. 0.5 is a coin flip, 1.0 is perfect. Used here for the fusion head.
Calibration / ECE	Whether a claimed probability is honest — if the model says 30%, does it happen 30% of the time? Expected Calibration Error measures the gap. It matters here because the fusion ledger <i>adds up</i> probabilities across repeated sightings, so a systematic bias compounds rather than averaging out.

Network anatomy — and what P2, P3, P4, P5 mean

A convolutional detector has three parts. The **backbone** repeatedly shrinks the image while extracting features. The **neck** recombines features from different depths. The **head** turns them into boxes and class scores.

As the backbone shrinks the image, it produces feature maps at several scales. These are named **P2, P3, P4, P5** — where **P_n** means the map has been reduced by a factor of **2ⁿ**. That factor is called the **stride**.

LEVEL	STRIDE	MAP SIZE FROM A 640 PX INPUT	SEES
P2	4	160 × 160	Very small objects — finest detail, most expensive
P3	8	80 × 80	Small objects
P4	16	40 × 40	Medium objects
P5	32	20 × 20	Large objects — coarsest, cheapest

So **"P3–P5"** (adopted) means the detector attaches its head to three levels, strides 8, 16 and 32. **"P2–P5"** (rejected) adds the stride-4 level as a fourth. Each level predicts independently, and the coarse levels handle big objects while the fine levels handle small ones.

Adding P2 is the textbook remedy for small-object detection. It is also expensive: the stride-4 map is **four times the area** of the stride-8 map, so it quadruples the number of candidate positions and roughly doubles total

computation. Part III shows it did not pay.

TERM	WHAT IT ACTUALLY MEANS
Anchor point	One candidate position on a feature map where the head may predict a box. P3–P5 at 640 px gives $80^2 + 40^2 + 20^2 = \mathbf{8,400}$. Adding P2 gives 34,000.
Anchor-free	YOLOv8 predicts a box directly from each position, rather than deforming pre-set template shapes as older detectors did.
DFL Distribution Focal Loss	Instead of predicting each box edge as a single number, the network predicts a <i>probability distribution</i> over 16 possible distances and takes its expected value. This lets it express uncertainty about a boundary — useful when the object is 12 px tall and its edge is genuinely ambiguous. It is also where quantisation broke the model (Part VII).
NMS Non-Maximum Suppression	Several anchors usually fire on the same object. NMS keeps the highest-scoring box and discards overlapping duplicates. Here it runs in NumPy on the Pi rather than inside the network.
Nano / the "n" in YOLOv8n	The smallest size in the family. Every layer's width and depth is scaled down. 3.0 M parameters against ~68 M for the largest.
GFLOPs	Billions of arithmetic operations for one forward pass — the honest measure of compute cost, independent of hardware. This model: 8.19 .
Transfer learning	Starting from weights trained on a large general dataset instead of from random. The detector starts from COCO; the hazard classifier from ImageNet. It is why a 6,000-image dataset is enough.
Fine-tuning	Continuing to train an already-trained network on new data, usually at a lower learning rate, to adapt it to a new domain without destroying what it knows.
Epoch	One complete pass through the training set.

Making it fit on a Raspberry Pi

TERM	WHAT IT ACTUALLY MEANS
Quantisation	Storing numbers in 8 bits instead of 32 to shrink the file and speed up arithmetic. The cost is precision, and the cost is not always small — see Part VII.
Weights vs activations	<i>Weights</i> are the learned numbers, fixed after training — they dominate file size. <i>Activations</i> are the intermediate values computed as data flows through — they dominate arithmetic. They can be quantised independently, and that distinction is the single most important engineering decision in this report.
Full INT8	Both weights and activations in 8 bits. Fastest, smallest — and here it destroyed the model, scoring 31.9% of reference.
w8a32 ADOPTED	8-bit weights , 32-bit float activations . Identical file size to full INT8 because weights dominate size, but the arithmetic stays in floating point. 99.2% of reference.
TFLite / LiteRT	A compact format and runtime for running trained models on small devices. The runtime is ~3 MB against ~600 MB for full TensorFlow — which is what makes 1 GB of RAM workable.
Tensor arena	The single block of memory LiteRT reserves for intermediate values. Allocated once at load, never per frame. Measured here at 8.39 MB.
GSD Ground Sample Distance	How much ground one pixel covers. $\text{altitude} / \text{focal_px}$. At 20 m here: 14.7 mm per pixel . It converts a real person into a pixel count, which is what determines whether the detector can see them at all.
NETD	The smallest temperature difference a thermal sensor can distinguish. The MLX90640 is 0.1 K — good enough that scene clutter, not sensor noise, is the limiting factor.

Part I · Design constraints

The architecture is not a general-purpose design. It is the consequence of four hard constraints, and every significant decision traces back to one of them.

CONSTRAINT	CONSEQUENCE FOR THE MODEL
Raspberry Pi 4, 1 GB RAM ~905 MB usable after the OS	Rules out whole-frame inference at useful rates. Forces a nominate-then-confirm cascade, allocate-once tensor buffers, and NumPy post-processing rather than a second framework.
Battery-powered flight	Compute is energy. A 640×640 forward pass is 409,600 pixels of work; a 160×160 crop is 25,600. The gate nominates 2–6 crops, so the detector does roughly 1/16 to 1/3 of a whole-frame detector's work.
Cost ceiling ≈ ₹14,000	Thermal sensing is an MLX90640 — 32×24 pixels , not a radiometric core. At a 20 m survey altitude a prone adult is 2.6 thermal pixels . The thermal path can therefore only ever be a <i>detector of anomalies</i> , never a classifier of people.
Search-and-rescue, not surveillance	A missed casualty is far worse than a false alarm, so the gate is deliberately permissive and the burden of rejection falls on later, more expensive stages. It also means calibrated probabilities matter more than raw ranking — an operator must be able to trust the number.

Part II · System architecture

Four stages, ordered by cost. Each runs only on what the previous stage passed.

STAGE A	STAGE B	STAGE C	STAGE D
Thermal gate Robust z-score over a 32×24 frame. Not machine learning — a statistical contrast test against the frame's own median and MAD. every frame · ~0 ms	Crop detector YOLOv8n over 160 px (or 224 px for large blobs) crops centred on gate nominations. The only heavy computation in the system. 2–6 crops/frame · 3.5 ms each	Scene classifier MobileNetV2 over the whole visible frame at 1 Hz. Supplies disaster context, not detection. 1 Hz · whole frame	Fusion ledger Logistic regression over 18 features, accumulating log-odds across repeated observations of the same ground location. per candidate · <0.1 ms

Stage A — why the gate is not a neural network

A 32×24 thermal frame contains 768 pixels. A prone adult at 20 m occupies roughly 2.6 of them. There is no learnable texture at that scale — only *contrast*. The gate therefore computes a robust z-score per pixel against the frame's own median and median-absolute-deviation, thresholds at `z_t = 2.5`, and groups survivors by 8-connectivity. Parameters live in `configs/pipeline.yaml` and are never hard-coded:

```
gate:
  sigma_min_k: 0.15      # noise floor in kelvin; MLX90640 NETD is 0.1 K RMS
  z_t: 2.5               # nomination threshold
  dt_min_k: 0.5          # minimum absolute thermal contrast
  two_sided: true        # cold anomalies count too (water, voids)
  max_blob_px: 30
  budget: {z_lo: 1.5, z_hi: 4.0, n_min: 2, n_max: 6}
```

Two-sided is deliberate. A body in water, or a void beneath rubble, reads *colder* than its surroundings. A one-sided "find hot things" gate would miss exactly the casualties that are hardest to find by eye.

The budget clamps crop count, not confidence. Between `n_min = 2` and `n_max = 6` crops are emitted per frame regardless of how many blobs pass, so the detector's per-frame cost is bounded no matter what the

scene does. This is what makes the latency predictable enough to fly.

Stage B — the detector, in detail

The detector is a **YOLOv8-nano** with the P3–P5 detection head. Measured from the shipped checkpoint:

PROPERTY	VALUE	MEANING
Parameters	3,011,628	Trainable weights
Modules	226	Backbone + neck + head
GFLOPs @ 640	8.19	Whole-frame fallback cost
Detection levels	3	Strides 8, 16, 32
Anchor points @ 640	8,400	$80^2 + 40^2 + 20^2$
Anchor points @ 160	525	The deployed crop path
Classes	4	Person, Car, Bicycle, OtherVehicle
reg_max	16	DFL bins per box side

Anchor-free, with distributional box regression

YOLOv8 predicts, at every anchor point, four *distributions* rather than four numbers. Each box side is represented as a softmax over `reg_max = 16` bins, and the final distance is the expectation of that distribution. This Distribution Focal Loss formulation lets the network express uncertainty about an edge — valuable when the object is 12 pixels tall and its boundary is genuinely ambiguous.

This detail is not academic — it is where quantisation broke the model

The DFL expectation is a weighted sum over 16 softmax bins, followed by a division by the input size to normalise. When *activations* are quantised to 8 bits, that normalisation constant is quantised too. Measured on the shipped INT8 file, the effective divisor was **≈636 instead of 640** — a 0.6% error that shifted every box centre by ~2.5 px. See Part V.

Why nano, and why 160 px crops

The `n` scaling is load-bearing, not a default. The crop path's job is not hard: by the time a crop reaches the detector, the gate has already asserted that something anomalous is centred in it. The detector answers a nearly binary question — *person or not* — on a well-framed 160 px patch. Pixels-on-target for the crop path, computed from $\text{GSD} = \text{altitude} / \text{focal_px} = 20 / 1359.3 = 14.7 \text{ mm/px}$:

PATH	PRONE ADULT	STANDING ADULT	ASSESSMENT
Thermal 32×24	2.6 px	~0.7 px	Anomaly detection only
RGB 160 px crop DEPLOYED	115×31 px	31×24 px	Comfortably resolvable
RGB 640 px full frame	45×12 px	12×9 px	Genuinely small-object

The crop path is **not** a small-object problem. This single table is why the P2 architecture was rejected — see Part III.

Part III · Architecture selection: P3 vs P2

Two architectures were built, trained under identical conditions, and measured. This section documents the one that was rejected, because the reasoning is the most defensible part of the design.

The older/alternative architecture: P2–P5 REJECTED

The standard remedy for small-object detection is to add a higher-resolution detection level. The P2 variant adds a **stride-4** head to the existing stride-8/16/32 levels:

PROPERTY	P3–P5 ADOPTED	P2–P5 REJECTED
Strides	8, 16, 32	4, 8, 16, 32
Detection levels	3	4
Modules	226	278
Anchor points @ 640	8,400	34,000
GFLOPs @ 640	8.86	17.43
Parameters	3,157,200	3,354,144

Note that P2 adds only 6% more parameters but **97% more computation** and **four times the anchors**. The stride-4 feature map is 160×160 at a 640 px input — it is large, and every operation on it is expensive.

The measured comparison

Both variants: 30 epochs, HIT-UAV, imgsz 640, identical seed, identical augmentation, identical optimiser. The only difference is the head.

METRIC	P3	P2	VERDICT
Person AP ₅₀	0.8211	0.8010	P3 by 2.5%
Overall mAP ₅₀	0.6387	0.5880	P3 by 8.6%
AP ₅₀ — objects 8–16 px	0.657	0.612	P3
AP ₅₀ — objects 16–32 px	0.772	0.753	P3
AP ₅₀ — objects 32+ px	0.814	0.791	P3
Compute	8.86	17.43	P3 at half

P2 lost in every object-size bin, including the smallest — the bin it exists to improve — at twice the compute. This is not noise; the ordering is consistent across all three bins and both aggregate metrics.

Why the intuitive choice failed

Three compounding reasons:

- **The resolution was never the bottleneck.** HIT-UAV objects are 8–32 px. Stride 8 already samples a 12 px person across 1.5 feature cells; stride 4 samples it across 3. The extra detail is real but marginal, and it is detail the nano backbone has too little capacity to exploit.
- **Four times the anchors is four times the optimisation difficulty.** With 34,000 anchor points and a comparable number of ground-truth boxes, the positive/negative ratio collapses. The task assignment problem becomes materially harder for no additional signal.

- **The deployment path is 160 px crops, not 640 px frames** — where, per the pixels-on-target table, a prone adult is 115×31 px. P2's advantage, had it existed, would have applied to the fallback path only.

P2 also would not train on Apple Silicon with default settings

Two failures at once, both caused by the 34,000-anchor loss computation:

- **fp16 overflow.** `cls_loss` ran $4.97 \rightarrow 46 \rightarrow -1224$ by iteration 6. A classification loss cannot be negative; the accumulator had overflowed.
- **Memory exhaustion.** 8.15 GB allocated on an 8 GB machine, driving the system into swap and slowing iteration from 2.1 s to 466 s — a 220× degradation.

Resolved with `--amp False --batch 6`. The comparison remains fair because Ultralytics accumulates gradients to a nominal batch of 64 regardless of the micro-batch: batch 16 $\rightarrow$ 64, batch 6 $\rightarrow$ 66.

Part IV · Training data

Three datasets, used for three different purposes. Two of them are thermal; the third teaches scene context.

DATASET	MODALITY	TRAIN	VAL	ROLE
HIT-UAV	Thermal, aerial nadir	2,029	290	Primary detector training. Clean overhead campus scenes, 4 classes.
RGBTDronePerson (thermal half)	Thermal, aerial oblique	4,900	1,225	Second domain: urban, night, dense. Person only.
RGBTDronePerson (visible half)	Visible, night	4,900	1,225	Abandoned — see Part X
AIDER	Visible, mixed aerial	6,433	total	Scene hazard classifier. 5 classes.

The combined thermal training set

The adopted model is trained on **HIT-UAV + RGBTDronePerson(thermal) merged**: 6,929 training and 1,515 validation images. The merge is non-trivial in two respects.

Class compatibility

RGBT labels are all class 0 (`person`); HIT-UAV's class 0 is `Person`. The label sets are therefore *already compatible*, which allowed training to retain the pretrained 4-class detection head rather than rebuilding it for a single class. Reducing to `nc=1` would have discarded the class branch's learned weights for no benefit.

The accepted cost, stated plainly

RGBT frames contain cars and bicycles that are **not labelled**, because that dataset annotates only people. Those become negative examples, so Car, Bicycle and OtherVehicle AP degrade. This is an accepted trade — the payload acts only on Person — but it means **overall mAP₅₀ must not be quoted from this model**. Per-dataset Person AP₅₀ is the honest metric.

Filesystem construction

Images are **hard-linked**, not copied or symlinked, consuming no additional disk. Symlinks fail here: Ultralytics locates labels by substituting the literal path component `/images/` with `/labels/`, and resolves symlinks *before* that substitution, turning the path back into `images_thermal/`. Source prefixes (`hit_`, `rgbt_`) are preserved in filenames so the validation set can be scored per-source.

Part V • Training methodology

Training proceeds in two stages, both on MPS (Apple Silicon GPU) on an 8 GB M1.

STAGE	INITIALISATION	DATA	EPOCHS	BATCH
1 — Base	COCO yolov8n.pt	HIT-UAV	30	16
2 — Domain merge	Stage 1 best.pt	Combined thermal	20	12

Hyperparameters, as actually used

```
optimizer: AdamW      lr0: 0.001      lrf: 0.01
momentum: 0.937      weight_decay: 0.0005
warmup_epochs: 3      patience: 8      seed: 0
box: 7.5      cls: 0.5      dfl: 1.5      # loss weights
imgsz: 640      device: mps      amp: True      # P3 only; P2 requires amp False
```

Augmentation — and the two settings that are deliberately zero

```
hsv_h: 0.0      <-- disabled on purpose
hsv_s: 0.0      <-- disabled on purpose
hsv_v: 0.3
degrees: 180.0
translate: 0.1      scale: 0.5     fliplr: 0.5
mosaic: 1.0      close_mosaic: 10
```

Why hue and saturation augmentation are switched off

A thermal image is a **single-channel intensity map**, replicated across three channels for the network's benefit. Its "colour" is not colour — it is temperature. Perturbing hue or saturation would make the three channels disagree, producing an image that corresponds to no physical scene and teaching the network to be invariant to a signal that carries the entire meaning. Brightness (`hsv_v`) is retained at 0.3, because that *does* have a physical analogue: a scene that is uniformly warmer or cooler.

Why rotation is the full $\pm 180^\circ$

A nadir aerial view has **no canonical orientation**. A person lying north-to-south and one lying east-to-west are the same object under a rotation of the sensor, and the drone's yaw is arbitrary. Constraining rotation would teach a spurious prior. This is the opposite of the correct setting for ground-level photography, where gravity fixes "up".

Part VI · Results

Detector, per dataset

Scored Person-only, at IoU 0.5, through identical letterboxing and identical NMS for every row. 590 images, 5,213 ground-truth people.

MODEL	HIT-UAV AP ₅₀	RGBT-THERMAL AP ₅₀	ASSESSMENT
Stage 1 — HIT-UAV only, 30 ep	0.8229	0.2752	Strong on one domain, poor on the other
Stage 2 — combined, 20 ep	0.8092	0.5231	Generalises, but had not converged
Stage 3 — combined, +25 ep ADOPTED	0.8499	0.5443	Better on BOTH than anything prior
Stage 3 vs Stage 1	+0.0270	+0.2691	No trade-off remains

The 20-epoch run appeared to cost 1.7% on the original domain. It had simply not converged — it was still improving when it stopped. Twenty-five further epochs recovered that and more: **the adopted model beats the HIT-UAV-only baseline on HIT-UAV itself** while nearly doubling accuracy on an independently collected dataset. **Training across two domains cost nothing.**

This is the defensible claim, and it is worth more than the raw number: a single-dataset result always invites the question of whether the network learned *people* or learned *that dataset's* particular sensor, altitude and scene statistics. Two independently collected datasets answer it before it is asked.

How the score behaves as the box requirement tightens

AP₅₀ requires only 50% overlap. Reporting it alone flatters any detector, so the full curve is given here.

OVERLAP REQUIRED (IOU)	AP	INTERPRETATION
0.50 — the standard metric	0.8499	Finds the person
0.60	0.7139	
0.70	0.4291	Fairly tight box
0.75	0.2342	Tight box
0.90	0.0011	Near pixel-perfect
AP₅₀₋₉₅ (mean over 0.50:0.95)	0.3712	The strict aggregate

Read this honestly: the model finds people reliably but boxes them loosely

For this mission that is the correct trade. The system must answer "*is there a person, and at what ground coordinate*" — not trace a silhouette. At the 20 m survey altitude the ground sample distance is 14.7 mm/px, so a box that is 5 px loose is **7 cm on the ground**. No rescue outcome depends on 7 cm. In a medical segmentation or an autonomous-driving stack, an AP₅₀₋₉₅ of 0.371 would be a defect; here it is capacity deliberately spent on size and speed instead.

Pixels-on-target curve

Recall as a function of the ground-truth person height in pixels, measured by progressively downsampling the validation set and padding back to the original canvas — so the network sees *smaller people*, not smaller images.

Padding uses the frame's own median, never black: on a thermal image, black is not neutral filler, it is a large very cold region.

PERSON HEIGHT	SAMPLES	RECALL
4–8 px	3	0.333
8–16 px	242	0.657
16–32 px	880	0.772
32+ px	43	0.814

This curve is not decorative — it is integrated to produce the Koopman sweep width W , the constant the entire search-coverage model rests on.

Scene hazard classifier

CLASS	PRECISION	RECALL	F1	SUPPORT
collapsed_building	0.797	0.979	0.879	96
fire	0.938	0.981	0.959	107
flooded_areas	0.892	0.958	0.924	95
traffic_incident	0.896	0.915	0.905	94
normal	0.988	0.949	0.968	894
Overall accuracy			0.952	1,286

Architecture: ImageNet-pretrained MobileNetV2 with `include_top=False`, frozen, plus a new classification head. Trained in two phases: head-only at `1e-3`, then the top third of the backbone unfrozen at `1e-4`. BatchNorm layers are held in inference mode throughout fine-tuning — updating running statistics from 32-image batches on a 6,433-image dataset is a reliable way to make validation accuracy collapse while training accuracy climbs.

Class weighting is used here, unlike in the fusion head. AIDER is 68% `normal`; an unweighted model scores 68% by answering "normal" every time.

Fusion head — the weakest component

A logistic regression over 18 features, deliberately not something larger. Its output must be a *calibrated probability*, because the ledger accumulates `logit(p)` across repeated observations of the same ground location; a miscalibrated score compounds. Its weights must also be readable, so an operator can be told *why* a candidate was flagged.

METRIC	VALUE	READING
ROC AUC	0.670	Weak ranking
Average precision	0.112	Weak, vs 0.031 base rate
Expected calibration error	0.0058	Well calibrated
Brier score	0.0243	vs 0.0247 base-rate baseline

Class balancing is deliberately NOT applied, unlike the hazard classifier. The ledger accumulates log-odds across passes; a balanced fit would bias every increment in the same direction, and that bias compounds with each observation rather than averaging out.

Five of eighteen features are structurally zero and are recorded as such rather than quietly dropped: `p_rgb`, `has_rgb`, `a_rgb` (the visible detector is unusable — Part X), `iou`, `d_c` (require the thermal↔RGB registration file, which needs both sensors physically mounted), and `hits`, `hit_ratio`, `alt_band` (unmeasurable from a still-image dataset; they require flight).

Part VII · Deployment and the quantisation study

The model that flies is not the model that is trained. It is converted to LiteRT (TFLite) and loaded by a ~3 MB runtime rather than a ~600 MB TensorFlow install. **Whether those two models are the same model had never been measured** — so an evaluation was built (`training/eval_int8_gap.py`) that compares them directly, through the same letterbox and the same decoder, so that the only variable is the numerics.

What it found

VARIANT	SIZE	PERSON AP ₅₀	% OF REFERENCE	MS/FRAME
PyTorch float32 (reference)	—	0.7252	100.0%	—
TFLite fp32	12.26 MB	0.7252	100.0%	89.0
TFLite w8a32 ADOPTED	3.32 MB	0.7193	99.2%	40.4
TFLite w8a16	3.38 MB	0.7208	99.4%	1064.8
TFLite full INT8 REJECTED	3.33 MB	0.2315	31.9%	25.1

That fp32 TFLite reproduces PyTorch to four decimal places confirms the conversion chain itself is exact. The loss is entirely attributable to quantisation.

Diagnosis of the INT8 failure

Class scores were *fine* — raw class-row correlation 0.987. The damage was confined to box regression, and it was **systematic rather than random**:

```
box error (int8 - float32), pixels at 640:
cx:  mean +2.460  median +2.715  std 2.293
cy:  mean +1.561  median +1.539  std 1.791
w :  mean -0.811  median -0.737  std 1.899
h :  mean -0.336  median -0.185  std 1.845
```

Quantisation *noise* is zero-mean. A consistent positive centre offset is not noise. Solving for the scale factor that best maps the INT8 output onto the float output gave **K = 636.06 for cx and 636.03 for cy** — identical to two decimal places across both axes. The graph's internal `1/640` normalisation constant had itself been quantised, to an effective `1/636`.

Correcting the scale recovers AP from 0.2315 to 0.5312 — so roughly half the loss is that constant, and half is residual activation noise. At 9–45 px people, a surviving 1–2 px error is most of an IoU-0.5 match.

Why w8a32 is the correct choice

w8a32 quantises the weights to int8 but leaves activations in float32. The file is the same 3.3 MB — weights dominate size — but the arithmetic that computes the DFL expectation and the normalisation stays in floating point. w8a16 is equally accurate and **26× slower**, because no optimised int16 kernels exist for these operations. fp32 is 3.7× the size for 0.8% more accuracy.

Post-fix verification

```
Person AP50  float32 (PyTorch)  0.8229
Person AP50  w8a32  (deployed) 0.8237  +0.1%
VERDICT: export is faithful
```

Measured ON the Raspberry Pi — not extrapolated

Every latency figure above is host-measured. On 11 September 2026 the stack was deployed to the target board and benchmarked there. This is the first execution of any part of this project on the hardware that flies.

OPERATION	MEDIAN	P90
Detector, 160 px crop THE DEPLOYED PATH	28.2 ms	28.3 ms
Detector, 224 px crop (large blobs)	56.7 ms	56.9 ms
Hazard classifier, 640×480 frame	33.0 ms	33.1 ms

All three models resident simultaneously: 95.2 MB peak RSS, leaving 730 MB of 905 free. SoC temperature 42.8 °C and throttled=0x0 — roughly half the 80 °C soft-throttle point, running from the payload's own power bank.

The frame budget closes

The thermal sensor delivers full frames at 4 Hz, giving 250 ms per frame:

gate (NumPy over 32x24)	~0 ms	
6 crops x 28.2 ms	169 ms	<- worst case
hazard, amortised at 1 Hz	8 ms	

	177 ms	-> 29% headroom

The typical case — two nominated crops — is **56 ms**, leaving the processor idle 78% of the time.

This measurement validates the entire architecture

The host-to-Pi ratio measured here is ~**12×**. A whole-frame 640 px pass costs 27 ms on the development machine, which extrapolates to roughly **320 ms on the Pi** — over the entire 250 ms frame budget before any other work is done. **A conventional whole-frame detector does not fit on this hardware at this frame rate. The nominate-then-confirm cascade does.** That is no longer an argument; it is a measurement.

Software footprint on the Pi

ITEM	MEASURED
Detector model file	3.23 MB
Hazard model file	2.74 MB
Fusion head	1.3 KB
TFLite tensor arena	8.39 MB
Whole Python process	38.2 MB
Available on the Pi 4	~905 MB
Inference, 160 px crop	3.5 ms

Input tensors are **NCHW float32**, and box regressions are **normalised to [0,1]** — both measured from the file rather than assumed, because neither matches the common documentation and neither raises when assumed wrongly. A pixels/normalised confusion scales every box by the input size and still returns plausible geometry. The runtime therefore *probes* the box convention at load time.

Part VIII · Airframe integration

The payload is designed to attach to a **generic quadcopter**, not to a specific airframe. It is deliberately a self-contained module rather than an avionics component.

The physical interface

PROPERTY	VALUE	NOTE
Tray footprint	145 × 135 × 9 mm	Flat plate, printed PLA/PETG
Assembled height	~35.5 mm	With the power bank fitted
Printed mass	29.8 g	Tray + GPS mast
Total payload mass	~340 g	Bank 209 g, Pi 46 g, sensors and cabling ~55 g
Primary attachment	4 × M3	On a 133 × 123 mm rectangle, one per corner
Secondary attachment	2 × 4.5 mm slots	Mid-edge, for a zip tie or hook-and-loop strap

The corner M3s sit **on the fillet arc centres**, which gives a uniform 4.3 mm of material around each bolt in every direction — these four screws carry the whole payload, and an earlier inset-from-the-edges placement left them *breaching* the rounded corner by 0.53 mm.

How it attaches in practice

Few airframes will happen to have a 133 × 123 mm bolt pattern, so the intended fitment is the standard two-layer approach used for any drone payload:

1. **A damping plate between the airframe and the tray.** Four rubber ball dampers (M3, the common 20–30 mm spacing) isolate the payload from motor vibration. **This is not optional:** the MPU-6050 provides the attitude solution that projects detected pixels onto ground coordinates, and an undamped IMU on a quadcopter produces attitude noise that turns directly into GPS error on every detection.
2. **The tray bolts to the damping plate** via the four corner M3s, or straps to the airframe's existing battery rails through the two zip-tie slots. The slots exist precisely because strap mounting requires no matching hole pattern at all.
3. **The GPS mast is a separate part**, mounted above and away from the Pi. A Pi 4 radiates broadband RF right across the GPS L1 band at 1575 MHz — the HDMI serialiser especially — and a patch antenna sitting beside it loses satellites. The symptom is a slow or wandering fix rather than an obvious failure, which makes it easy to misdiagnose.

What airframe it needs

At 340 g, the payload suits a **450 mm-class quadcopter or larger** (an F450/S500 frame with 2212-class motors, or any craft rated for 500 g of payload). A 5-inch or racing-class quad is unsuitable. The conventional rule of a 2:1 thrust-to-weight ratio at all-up weight should be maintained; below that, stability in wind degrades and endurance collapses.

The **flat rather than stacked** layout is a deliberate consequence: the 209 g power bank dominates the mass budget, and stacking it would raise the centre of gravity and make the payload pendulum beneath the airframe. Flat keeps the CG low and close to the mounting plane, and prints with no supports.

The integration property that matters most: it is electrically independent

The payload has **its own battery, its own GPS and its own IMU**, and makes **no electrical connection to the flight controller**. It cannot brown out the flight stack, cannot inject noise onto a shared bus, and cannot contribute to a flyaway. Fitting it to a new airframe is a mechanical operation, not an avionics integration — no firmware change, no re-tuning, no recertification. That is what makes "works on a generic quadcopter" a real claim rather than a hopeful one.

Part IX · What this adds to a drone

The honest framing: this payload makes a drone **perceptually autonomous today**, and **navigationally autonomous with one cable** that is not yet fitted.

The levels, and where this sits

LEVEL	WHAT IT MEANS	STATUS
L0 Manual	A pilot flies; a human watches a live video feed and calls out anything they see. This is how most drone search is done today.	The baseline being replaced
L1 Perceptual autonomy	The pilot (or an existing waypoint mission) flies the pattern. The payload independently detects, confirms, fuses evidence across passes, geo-tags each candidate and logs it. No human watches the feed. No connection to the flight controller.	Built and deployed
L2 Closed-loop tasking	The payload talks to the flight controller over MAVLink and re-tasks the aircraft — orbiting a high-confidence candidate for a second look, or densifying the search pattern where the coverage model says probability is concentrated.	One UART cable away. Not built.
L3 Mission autonomy	The payload plans the search itself from a probability map, and adapts it in flight.	Out of scope

Why L1 is the part that actually matters

The bottleneck in drone-based search is **not flying the aircraft** — waypoint missions have been solved for a decade. The bottleneck is **a human being watching a thermal video feed**. Sustained visual search degrades sharply with time on task; a 2.6-pixel warm blob passing across a low-resolution thermal image at 4 Hz is close to the worst case for human attention. The operator also cannot rewind, cannot accumulate weak evidence across three separate passes over the same rubble, and cannot attach a GPS coordinate to what they thought they saw.

This payload does all four. Specifically, it contributes:

- **Detection without supervision.** The aircraft can fly its pattern with nobody watching the feed, or beyond visual line of sight entirely.
- **Evidence that accumulates.** The fusion ledger adds log-odds for the same ground location across repeated passes, so three weak sightings compound into one confident detection. A human watching a live feed cannot do this at all.
- **A coordinate, not an impression.** Every candidate carries a GPS fix, computed by projecting the detection through the IMU attitude solution — which is why the damping plate in Part VIII is a detection-accuracy component, not a comfort feature.
- **A calibrated probability.** The operator is given a number they can triage on, not a bounding box. This is why the fusion head is a logistic regression whose weights can be read and explained, rather than something larger and opaque.

- **A quantified search.** Because coverage is modelled (Koopman: $POD = 1 - e^{-C}$, $C = W \cdot L/A$), the system can report *how thoroughly an area was actually searched* — which is what lets a coordinator decide to move on rather than guess.

The path to L2, and why it is deliberately not built yet

The Pi's UART is already free for it: `dtoverlay=disable-bt` is applied and `/dev/ttyAMA0` is unclaimed, verified with `fuser`. A single three-wire connection to the flight controller's TELEM port would carry MAVLink.

It is not built because **the moment the payload can command the aircraft, it becomes flight- safety-critical**. Every failure mode above — a gate that fires on hot roofing, a detector confidently wrong on an unfamiliar scene — stops being a false alarm in a log and starts being an unplanned manoeuvre. That step should be taken after the perception stack has been validated in the field, not before. The current design's electrical independence is the property that makes L1 deployable now with no safety case at all.

Part X · Failure analysis

Four failures are documented here because each produced a working-looking system with wrong numbers, and none raised an error. They are the strongest evidence that the validation described above is necessary rather than ceremonial.

1. The visible-light detector could never have worked

A fine-tune on RGBTDronePerson's visible frames reached **mAP₅₀ 0.007** — precision 1%, recall 24% — across three separate runs (8 epochs at half data, then 25 epochs at full data). Each failure was treated as a training problem.

The cause was the data. RGBTDronePerson's visible frames are **night images**: 32% have a mean brightness below 40/255. The labelled people are not visible in the visible channel at all.

The decisive test — running the HIT-UAV thermal model, with no fine-tuning, against the same labels:

FRAMES	RECALL @ IOU 0.5	DETECTIONS
Thermal	49.9% (456/914)	1,540
Visible	0.0% (0/914)	692

Zero matches from 692 detections is not "difficult" — it is "absent". The test also proves the labels are correct: a detector that had never seen this dataset finds half of them on the thermal frames. **No training configuration could have succeeded.**

Method worth generalising: when a model will not learn, verify the signal is present in the data *before* adjusting hyperparameters. Running an existing unrelated model against the labels costs minutes. Here it would have saved days.

2. The hazard classifier answered "normal" to every image ever shown to it

Including AIDER's own `fire/` and `collapsed_building/` folders, which the Keras model classifies at 97.9% recall.

The model was correct; the runtime was not. Two input conventions exist and this project ships both:

MODEL	INPUT TENSOR	GRAPH EXPECTS
Detectors (Ultralytics)	float32	0–1 normalised
Hazard (MobileNetV2)	uint8, scale 1.0, zp 0	raw 0–255

The hazard model keeps `preprocess_input` *inside* its graph. Feeding it `pixels/255` quantised to `rint(p/255)` — **0 below 128, 1 above** — a two-level near-black image. Three of the eighteen fusion features were identically zero, and the fusion head correctly assigned them zero weight, fitting around dead inputs.

Fix: read the units off the tensor. `scale × (qmax - zero_point)` is ≈ 1.0 for a normalised graph and ≈ 255 for a raw-pixel graph — a 255× separation, so the threshold is unambiguous. **Result: fusion ROC AUC 0.620 → 0.670, AP 0.089 → 0.112.**

3. A 97.8% accuracy that meant nothing

An earlier hazard classifier reported 97.8% validation accuracy. Its confusion matrix had four empty rows.

Keras's `shuffle=False` — added for reproducible report ordering — also disables shuffling of the *file list before the split*. Since `image_dataset_from_directory` takes the last 20% of a class-ordered list, and `normal` is the

final class holding 4,390 of 6,433 images, **the entire validation set was one class**. The model scored 97.8% by answering "normal".

The corrected figure is **95.2% across five populated classes** — a far more valuable number than the 97.8% it replaced.

4. Reporting code discarding completed training

The hazard script has twice crashed *after* training and quantisation completed — once destroying six consecutive runs of roughly 20 minutes each. Both crashes were in report generation, not in the model.

Structural fix: reporting is now a single `finish()` function reached by both code paths, with a `--report-only` flag that regenerates metrics from the saved model. A reporting failure can no longer cost a retrain.

Part XI • Known limits

Stated explicitly, because a report that lists only successes is not useful for deciding what to do next.

LIMITATION	STATUS
The fusion head is weak (ROC AUC 0.670)	Follows from five of eighteen features being structurally zero. Will improve when registration and flight data exist — not before.
Gate recall is measured out-of-domain — 2.9% on RGBT	A contrast gate tuned for rubble, evaluated on dense urban night scenes with ~10 people per frame. Not a valid figure for the mission. Must be re-measured on the actual MLX90640 over rubble.
Sweep width W is not published	Deliberately held back. It integrates gate recall; feeding the out-of-domain 2.9% figure would understate coverage by roughly 5×.
No visible-light detector	The system is thermal-only today. A daytime visible branch requires a daytime aerial dataset (VisDrone, SARD, HERIDAL) — scoped future work, not a broken component.
Thermal↔RGB registration not calibrated	Requires both sensors physically mounted. The code degrades safely without it.
Never flown	The stack is benchmarked on the target board but has not operated on an airframe in motion. Vibration, attitude dynamics and real thermal scenes are all unvalidated.
OtherVehicle AP₅₀ ≈ 0.05	A junk catch-all class. It is why Person AP ₅₀ is quoted rather than overall mAP ₅₀ .

Appendix • Reproduction

```
bash training/run_all.sh          # idempotent; skips completed steps

1 detector P3 baseline            HIT-UAV, 30 epochs
2 detector P2 (negative result) HIT-UAV, 30 epochs, --amp False --batch 6
3 pixels-on-target curve
4 thermal domain merge            combined, 20 epochs
5 LiteRT w8a32 export            160 / 224 / 640
6 hazard classifier               AIDER, 4 + 12 epochs
7 fusion dataset                 RGBT val, real gate
8 fusion head                    logistic regression
  eval_sweep_width              HELD BACK — RUN_SWEEP_WIDTH=1 to force
```

Environment. Two virtual environments are required and must not be merged. `.venv` (Python 3.13) runs the application, dashboard and tests. `.venv-tf` (Python 3.12) runs the entire ML pipeline, because TensorFlow publishes no macOS-arm64 wheel for Python 3.13 and the LiteRT export needs it.

The pinned versions fight each other. `torchvision` must match `torch` exactly or the validator dies at the end of epoch 1 with `RuntimeError: operator torchvision::nms does not exist; libtorch requires torch < 2.14`. The only combination satisfying both is **torch 2.13.0 + torchvision 0.28.0**. Re-check `pip list | grep torch` after any installation.

SaResQ perception stack · repository commit `4fe9173` · 167 automated tests passing · all figures measured on an Apple M1 (8 GB) unless stated.